

# TulipaEnergyModel.jl: A Modelling Framework Breaking the Tradeoff Between Fidelity and Computational Load

Abel Soares Siqueira<sup>1</sup>, Diego A. Tejada-Arango<sup>6,9</sup>, Grigory Neustroev<sup>7</sup>, Juha Kiviluoma<sup>4</sup>, Lauren Clisby<sup>8</sup>, Maaïke Elgersma<sup>3</sup>, Ni Wang<sup>2</sup>, Suvayu Ali<sup>1</sup>, Zhi Gao<sup>5</sup>, Mathijs M. de Weerd<sup>3</sup>, and Germán Morales-España<sup>2,3</sup>

<sup>1</sup> Netherlands eScience Center, Netherlands <sup>2</sup> TNO - Netherlands Organisation for Applied Scientific Research, Netherlands <sup>3</sup> Delft University of Technology, Netherlands <sup>4</sup> Nodal Tools, Finland <sup>5</sup> Utrecht University, Netherlands <sup>6</sup> Universidad Pontificia Comillas <sup>7</sup> Zeroth Research <sup>8</sup> Open Energy Transition <sup>9</sup> Electric Power Research Institute

DOI: 10.xxxxxx/draft

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: Marlin Arnz

## Reviewers:

- [@DavidPozo](#)
- [@trulsf](#)

Submitted: 20 October 2025

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0)

## Summary

TulipaEnergyModel.jl is a modelling framework for analysing investment and operational decisions of future energy systems through capacity expansion and dispatch optimisation. TulipaEnergyModel.jl is the main package of the Tulipa Energy ecosystem, and it is developed in Julia (Bezanson et al., 2017) using JuMP.jl (Lubin et al., 2023). As a framework, Tulipa formulates models entirely based on input data. This allows users to analyse virtually any system using the generalised building blocks – production, consumption, conversion, storage, and transport. TulipaEnergyModel.jl focuses on model quality and efficient implementation, allowing it to break the tradeoff between model fidelity and computational load through: tighter MIP formulations; exact LP reformulations with fewer constraints and variables; more accurate LP approximations; and flexible model fidelity across temporal, technological, and spatial dimensions.

## Statement of Need

Existing open-source frameworks for Energy System Optimisation include EnergyModelsX (Hellemo et al., 2024), PowerModels (Coffrin et al., 2018), SpineOpt (Ihlemann et al., 2022), Sienna ecosystem (Lara et al., 2021), GenX (Bonaldo et al., 2025), PyPSA (Brown et al., n.d.), Calliope (Pfenninger & Pickering, 2018), and AnyMOD (Göke, 2021). These frameworks have advanced the field through different modelling approaches, including flexible formulations. Nevertheless, increasing model detail can still create computational challenges in large-scale applications. Alongside advances in solvers and computing, improving mathematical formulations can help increase model fidelity while simultaneously solving faster than standard formulations. This insight inspired the development of TulipaEnergyModel.jl, with the core philosophy of advancing the state-of-the-art in formulation quality by: 1) lowering computational cost while maintaining model fidelity, by reducing the problem size (Tejada-Arango et al., 2025), and by creating tighter mixed-integer programs (MIP) (Morales-España et al., 2013). 2) increasing model fidelity without extra computational cost, e.g., by developing more accurate linear programming (LP) approximations (Elgersma et al., 2026; Gentile et al., 2016; Morales-España et al., 2022). Finally, 3) balancing computational burden with adaptive/flexible model fidelity, i.e., having different levels of detail in various parts of the model, in the temporal (Gao et al., 2025), technological (Morales-España et al., 2022) and spatial dimensions. These modelling strategies offer significant computational benefits, especially when handling large-scale problems: covering a continent with multiple energy carriers, and optimising over

decades while maintaining hourly resolution for key aspects (e.g., renewable generation). TulipaEnergyModel.jl had to be developed from scratch to be able to include all of these modelling breakthroughs, since they alter the foundation and structure of the model. Below, we present some of core modelling and software design innovations.

## Modelling Innovations

Two of the main innovations include:

1. Fully flexible temporal resolution (Gao et al., 2025) for the assets/ flows,
2. Direct connections between assets (Tejada-Arango et al., 2025).

The following example illustrates these concepts:

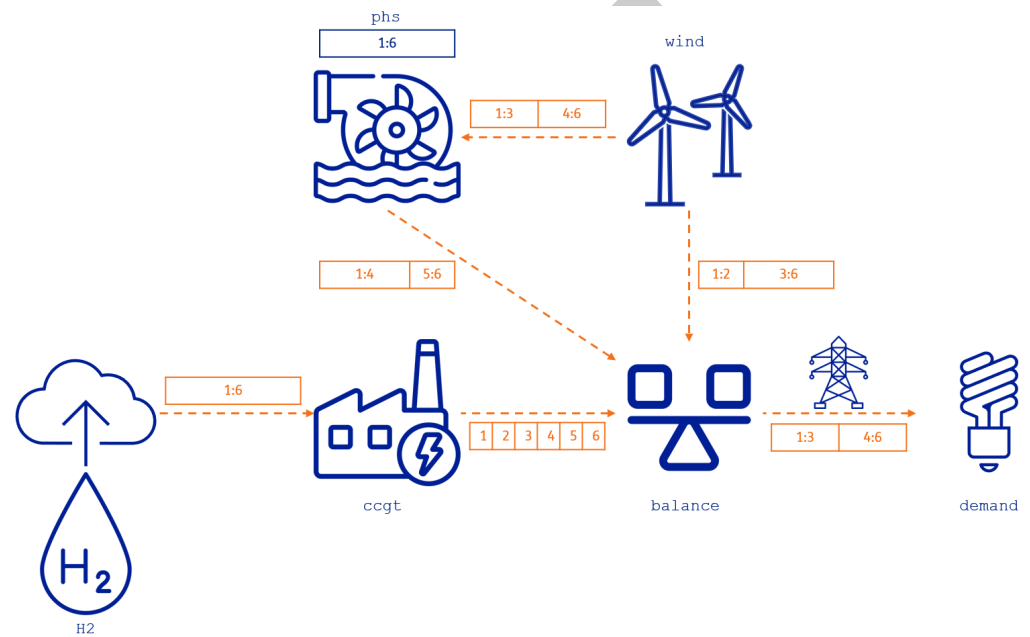


Figure 1: Example of network with flexible resolution of assets and flows

For the fully flexible temporal resolution, consider the 6-hour duration of this system. The flow between “H2” and “ccgt” has a resolution of 6 hours, while the flow between “ccgt” and “balance” is 1 hour. The resolution from “wind” to “phs” is 3 hours, and from “phs” to “balance” is irregular, a 4-hour block followed by a 2-hour block. Tulipa allows different temporal resolutions throughout the model, thereby reducing the number of variables and constraints. This feature can drastically speed up solving with little loss in accuracy (Gao et al., 2025).

For the direct connection between assets, the storage “phs” is directly connected to the “wind” to charge, and to “balance” to discharge. This direct connection between assets completely avoids intermediate elements (connections/nodes), thereby eliminating unnecessary variables and constraints. Thus, accelerating solving times without any loss of accuracy (Tejada-Arango et al., 2025).

TulipaEnergyModel.jl is fundamentally focused on high-quality mathematical formulations. The model also includes other key features such as seasonal storage modelling using representative periods (Neustroev et al., 2025; Tejada-Arango et al., 2018), tight and compact MIP formulations for storage (Elgersma et al., 2026), unit commitment (Morales-España et al., 2013), and compact formulations for multi-year investment (Wang & Morales-España, 2025; Wang & Tejada-Arango, 2025).

71 Although TulipaEnergyModel.jl is a relatively young package, it already shows promising results  
72 in breaking the trade-off between computational burden and model details. As presented in initial  
73 benchmark results, TulipaEnergyModel.jl allows to: 1) solve large capacity expansion problems  
74 faster (Neustroev et al., 2025), 2) include uncertainty through more computationally demanding  
75 stochastic programming (Kremer, 2025), 3) create computationally efficient approximations by  
76 exploiting the flexible temporal resolution (Gao et al., 2025), and 4) reduce the problem size  
77 without sacrificing any accuracy (Tejada-Arango et al., 2025). All these studies demonstrate  
78 how TulipaEnergyModel.jl does not rely on (over)simplifying the problem, leading to unreliable  
79 results/conclusions, to solve more computationally demanding problems.

80 **Software Design Innovations**

81 To accommodate flexible temporal resolution for assets and flows, many variables have a “time  
82 block” component instead of a “time step”, and many variables and constraints have sparse  
83 indices. To explain how this is implemented, revisit the system from Figure 1. In an hourly  
84 implementation, there would be variables such as  $f_{(H2,ccgt),t}$ ,  $f_{(phs,balance),t}$  for  $t = 1, \dots, 6$ .  
85 Instead, looking at the time blocks of each flow, there are  $f_{(H2,ccgt),1:6}$ ,  $f_{(phs,balance),1:4}$ , and  
86  $f_{(phs,balance),5:6}$ . In other words, this flow variable could be defined as

$$f_{e,b} \quad \forall e \in E, b \in B(e),$$

87 where  $E$  is the set of edges of the graph, and  $B(e)$  is the set of time blocks for each edge.

88 Similarly, some constraints and expressions have their own time blocks (e.g., the balance  
89 constraints), and obtaining the coefficients of the flows involves matching indices and computing  
90 the intersections between time blocks. Defining variables like this in JuMP leads to sparse  
91 indices, which are cumbersome and potentially slow. It can also create performance pitfalls,  
92 such as the “sum-if problem”, when trying to determine the coefficients of each flow in each  
93 balance constraint.

94 To improve the storage of these objects, Tulipa uses a tabular format to collect the indices of  
95 each variable, constraint, and expression. Each row of the table stores the corresponding set  
96 of indices for each JuMP object, and the JuMP objects themselves are stored in a linearised  
97 way in an array. Returning to the example system, the first three rows of the flow variable  
98 defined above are shown in Table 1.

Table 1: “Example of linearised tabular indices of the var\_flow table”

| id | from asset | to asset | time block start | time block end |
|----|------------|----------|------------------|----------------|
| 1  | H2         | ccgt     | 1                | 6              |
| 2  | phs        | balance  | 1                | 4              |
| 3  | phs        | balance  | 5                | 6              |

99 Storing the indices in tables avoids the sparse storage of JuMP objects, which can instead be  
100 stored in an array with their positions matching the IDs in the index tables. Figure 2 illustrates  
101 this method of storage.

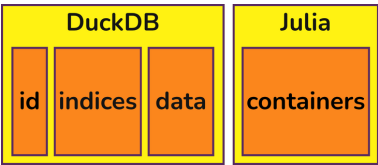


Figure 2: Representation of the storage of indices and JuMP objects

Another key design decision in Tulipa is handling data using a [DuckDB](#) ([Raasveldt & Muehleisen, n.d.](#)) connection - from processing input data, to generating internal tables for model creation, to storing outputs. This enables manipulating different data formats using DuckDB's capabilities (instead of Julia's), as well as decreasing data movement and duplication.

When creating variables and constraints, Tulipa can read the DuckDB tables row by row, reducing memory use compared to using an explicit Julia table, such as DataFrames.

Furthermore, using DuckDB for everything allows cleaner separation of the pipeline into (i) data ingestion and manipulation, (ii) indices creation, and (iii) model creation. Figure 3 summarises how Tulipa interacts with the DuckDB connection.

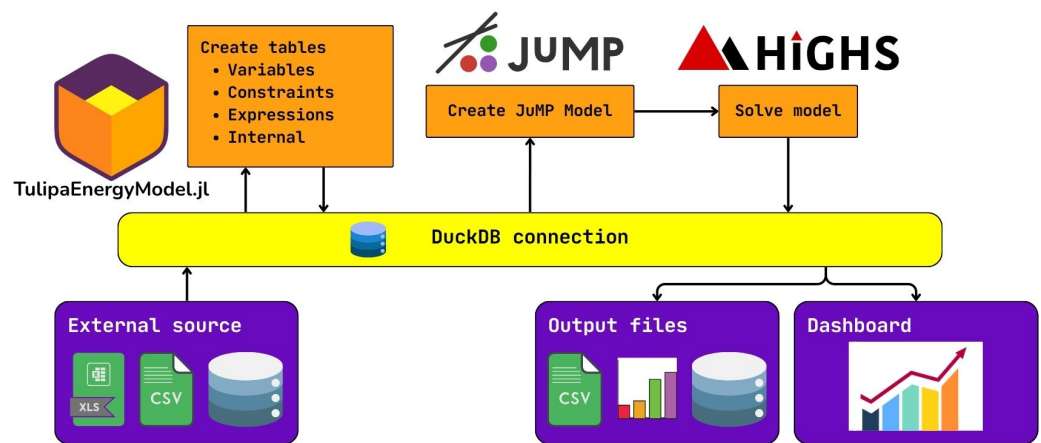


Figure 3: Overview of Tulipa's integration with DuckDB

First, users can prepare their data using any tools they prefer before loading it into the DuckDB connection. Then, the indices are created from the data in the DuckDB connection and saved back into it. While this step currently occurs in Julia, many of the operations use SQL and could potentially be moved outside Julia. Finally, the JuMP pipeline begins by reading the indices table and complementing the data as necessary, then creates the JuMP objects and generates the complete model, which is sent to the solver. Tulipa is tested and benchmarked using the HiGHS ([Huangfu & Hall, 2017](#)) solver, but other MIP solvers accepted by JuMP can be used as well.

### AI usage disclosure

We use and accept pull requests with AI coding assistants to help with development, but we expect the committers to understand and be responsible for the changes that they introduce. We expect contributors to include used coding agents as co-authors in their commits, such as:

Co-authored-by: MODEL NAME (FULL MODEL VERSION) <EMAIL>

No AI agents were used in writing this paper.

### Acknowledgements

We thank Oscar Dowson for the initial suggestion to use DataFrames, which motivated us to use tabular storage for all indices.

This publication is part of the project NextGenOpt ESI.2019.008, which is financed by the Dutch Research Council (NWO) and supported by the Netherlands eScience Center NLeSC C 21.0226. This project was also supported by TNO's internal R&D. This research is also (partially) funded by the European Climate, Infrastructure and Environment Executive Agency

under the European Union's HORIZON Research and Innovation Actions under grant agreement no. 101095998.

## References

- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Bonaldo, L., Chakrabarti, S., Cheng, F., Ding, Y., Jenkins, J., Luo, Q., Macdonald, R., Mallapragada, D., Manocha, A., Mantegna, G., Morris, J., Patankar, N., Pecci, F., Schwartz, A., Schwartz, J., Schivley, G., Sepulveda, N., & Xu, Q. (2025). *GenX* (Version 0.4.4). <https://doi.org/10.5281/zenodo.10846070>
- Brown, T., Hörsch, J., Hofmann, F., Neumann, F., Zeyen, L., Syranidis, C., Frysztacki, M., Schlachtberger, D., Glaum, P., & Parzen, M. (n.d.). *PyPSA: Python for Power System Analysis* (Version 0.35.1). <https://doi.org/10.5334/jors.188>
- Coffrin, C., Bent, R., Sundar, K., Ng, Y., & Lubin, M. (2018). PowerModels.jl: An open-source framework for exploring power flow formulations. *2018 Power Systems Computation Conference (PSCC)*, 1–8. <https://doi.org/10.23919/PSCC.2018.8442948>
- Elgersma, M. B., Morales-España, G., Aardal, K. I., Helistö, N., Kiviluoma, J., & Weerd, M. de. (2026). Tight MIP formulations for optimal operation and investment of storage including reserves. *IEEE Transactions on Power Systems*, 1–13. <https://doi.org/10.1109/tpwrs.2026.3669407>
- Gao, Z., Gazzani, M., Tejada-Arango, D. A., Soares Siqueira, A., Wang, N., Gibescu, M., & Morales-España, G. (2025). Fully flexible temporal resolution for energy system optimization. *Applied Energy*, 396, 126267. <https://doi.org/10.1016/j.apenergy.2025.126267>
- Gentile, C., Morales-España, G., & Ramos, A. (2016). A tight MIP formulation of the unit commitment problem with start-up and shut-down constraints. *EURO Journal on Computational Optimization*, 5(1), 177–201. <https://doi.org/10.1007/s13675-016-0066-y>
- Göke, L. (2021). AnyMOD.jl: A julia package for creating energy system models. *SoftwareX*, 16. <https://doi.org/10.1016/j.softx.2021.100871>
- Hellemo, L., Bødal, E. F., Holm, S. E., Pinel, D., & Straus, J. (2024). EnergyModelsX: Flexible energy systems modelling with multiple dispatch. *Journal of Open Source Software*, 9(97), 6619. <https://doi.org/10.21105/joss.06619>
- Huangfu, Q., & Hall, J. A. J. (2017). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142. <https://doi.org/10.1007/s12532-017-0130-5>
- Ihlemann, M., Kouveliotis-Lysikatos, I., Huang, J., Dillon, J., O'Dwyer, C., Rasku, T., Marin, M., Poncelet, K., & Kiviluoma, J. (2022). SpineOpt: A flexible open-source energy system modelling framework. *Energy Strategy Reviews*, 43, 100902. <https://doi.org/10.1016/j.esr.2022.100902>
- Kremer, L. A. A. (2025). *Stochastic programming for energy models: A blended cross-scenario representative periods approach* [Master's thesis, Delft University of Technology]. <https://resolver.tudelft.nl/uuid:0e87f306-1c92-4eec-805f-72377ed57fb2>
- Lara, J. D., Barrows, C., Thom, D., Krishnamurthy, D., & Callaway, D. (2021). PowerSystems.jl — A power system data management package for large scale modeling. *SoftwareX*, 15. <https://doi.org/10.1016/j.softx.2021.100747>
- Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023). JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. <https://doi.org/10.1007/s12532-023-00239-3>

- 177 Morales-España, G., Latorre, J. M., & Ramos, A. (2013). Tight and compact MILP formulation  
178 for the thermal unit commitment problem. *IEEE Transactions on Power Systems*, 28(4),  
179 4897–4908. <https://doi.org/10.1109/TPWRS.2013.2251373>
- 180 Morales-España, G., Martínez-Gordón, R., & Sijm, J. (2022). Classifying and modelling demand  
181 response in power systems. *Energy*, 242, 122544. [https://doi.org/10.1016/j.energy.2021.](https://doi.org/10.1016/j.energy.2021.122544)  
182 [122544](https://doi.org/10.1016/j.energy.2021.122544)
- 183 Neustroev, G., Tejada-Arango, D. A., Morales-Espana, G., & Weerdt, M. M. de. (2025). *Hull*  
184 *Clustering with Blended Representative Periods for Energy System Optimization Models*.  
185 arXiv. <https://doi.org/10.48550/arXiv.2508.21641>
- 186 Pfenninger, S., & Pickering, B. (2018). Calliope: A multi-scale energy systems modelling  
187 framework. *Journal of Open Source Software*, 3(29), 825. [https://doi.org/10.21105/joss.](https://doi.org/10.21105/joss.00825)  
188 [00825](https://doi.org/10.21105/joss.00825)
- 189 Raasveldt, M., & Muehleisen, H. (n.d.). *DuckDB*. <https://github.com/duckdb/duckdb>
- 190 Tejada-Arango, D. A., Domeshek, M., Wogrin, S., & Centeno, E. (2018). Enhanced  
191 representative days and system states modeling for energy storage investment analysis. *IEEE*  
192 *Transactions on Power Systems*, 33(6), 6534–6544. [https://doi.org/10.1109/TPWRS.](https://doi.org/10.1109/TPWRS.2018.2819578)  
193 [2018.2819578](https://doi.org/10.1109/TPWRS.2018.2819578)
- 194 Tejada-Arango, D. A., Kiviluoma, J., & Morales-España, G. (2025). Debunking the speed-  
195 fidelity trade-off: Speeding-up large-scale energy models while keeping fidelity. *International*  
196 *Journal of Electrical Power & Energy Systems*, 168, 110674. [https://doi.org/10.1016/j.](https://doi.org/10.1016/j.ijepes.2025.110674)  
197 [ijepes.2025.110674](https://doi.org/10.1016/j.ijepes.2025.110674)
- 198 Wang, N., & Morales-España, G. (2025). *Vintage-based formulations in multi-year investment*  
199 *modelling for energy systems*. <https://arxiv.org/abs/2505.00379>
- 200 Wang, N., & Tejada-Arango, D. A. (2025). *Discounting approaches in multi-year investment*  
201 *modelling for energy systems*. <https://arxiv.org/abs/2504.21709>